

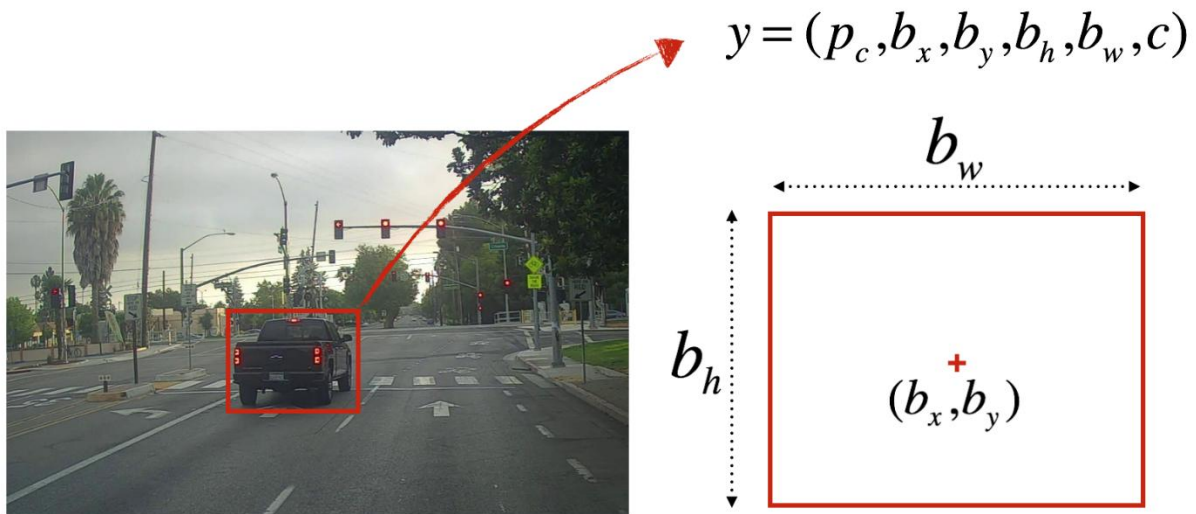
Autonomous Driving – Car Detection

In this assignment and the next, you'll implement object detection using the very powerful YOLO model.

Problem Statement

You are working on a self-driving car. As a critical component of this project, you'd like to first build a car detection system. To collect data, you've mounted a camera to the front of the car, which takes pictures of the road ahead every few seconds as you drive around.

You've gathered all these images into a folder and labelled them by drawing bounding boxes around every car you found. Here's an example of what your bounding boxes look like:



$p_c = 1$: confidence of an object being present in the bounding box

$c = 3$: class of the object being detected (here 3 for "car")

If there are 80 classes you want the object detector to recognize, you can represent the class label c either as an integer from 1 to 80, or as an 80-dimensional vector (with 80 numbers) one component of which is 1, and the rest of which are 0 (i.e., one-hot encoding). In this project, you'll use both representations, depending on which is more convenient for a particular step.

In this exercise, you'll discover how YOLO ("You Only Look Once") performs object detection, and then apply it to car detection. Because the YOLO model is very computationally expensive to train, the pre-trained weights are already loaded for you to use.

YOLO

"You Only Look Once" (YOLO) is a popular algorithm because it achieves high accuracy while also being able to run in real time. This algorithm "only looks once" at the image in the sense that it requires only one forward propagation pass through the network to make predictions. After non-max suppression, it then outputs recognized objects together with the bounding boxes.

Model Details

Inputs and outputs

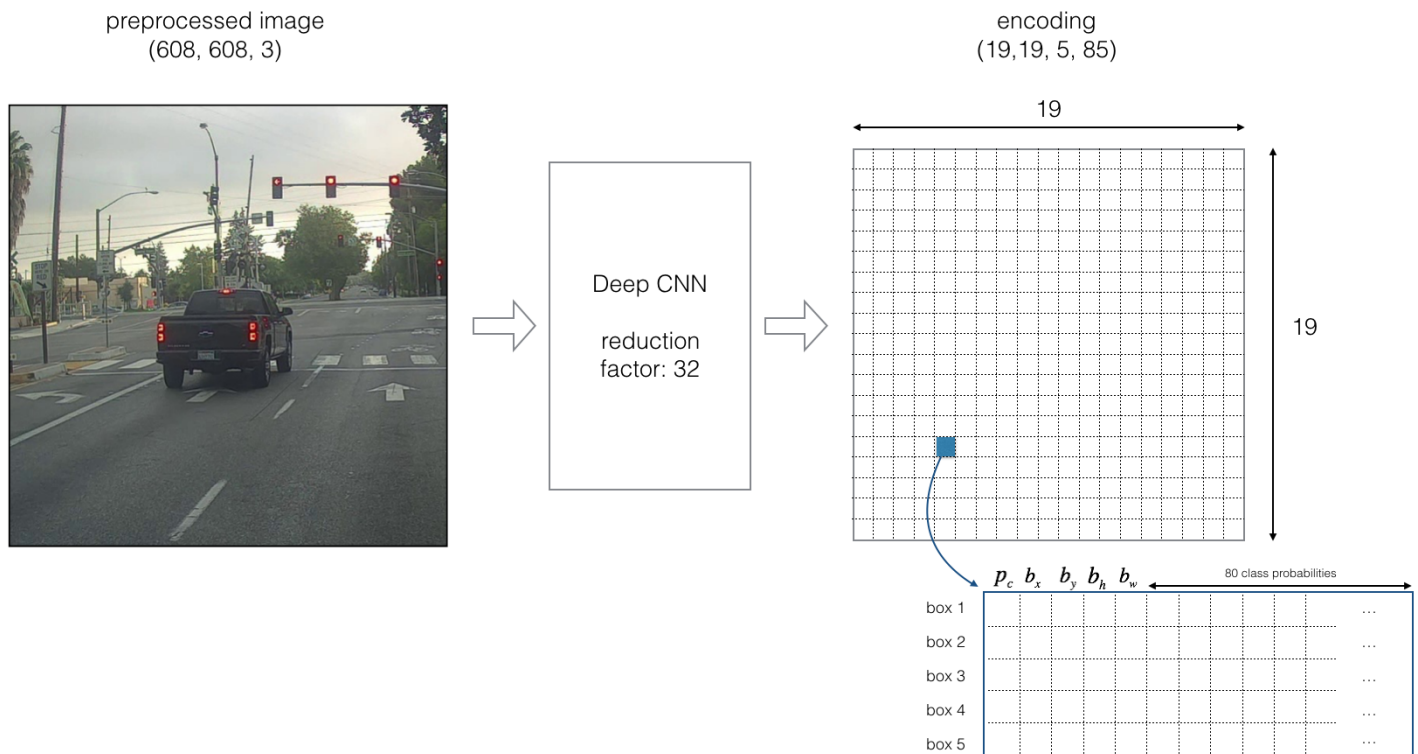
- The **input** is a batch of images, and each image has the shape (608, 608, 3)
- The **output** is a list of bounding boxes along with the recognized classes. Each bounding box is represented by 6 numbers ($p_c, b_x, b_y, b_h, b_w, c$) as explained above. If you expand c into an 80-dimensional vector, each bounding box is then represented by 85 numbers.

Anchor Boxes

- Anchor boxes are chosen by exploring the training data to choose reasonable height/width ratios that represent the different classes. For this assignment, 5 anchor boxes were chosen for you (to cover the 80 classes), and stored in the file yolo_anchors.txt
- The dimension of the encoding tensor of the second to last dimension based on the anchor boxes is (**m, nH, nW, anchors, classes**).
- The YOLO architecture is: IMAGE (m, 608, 608, 3) -> DEEP CNN -> ENCODING (m, 19, 19, 5, 85).

Encoding

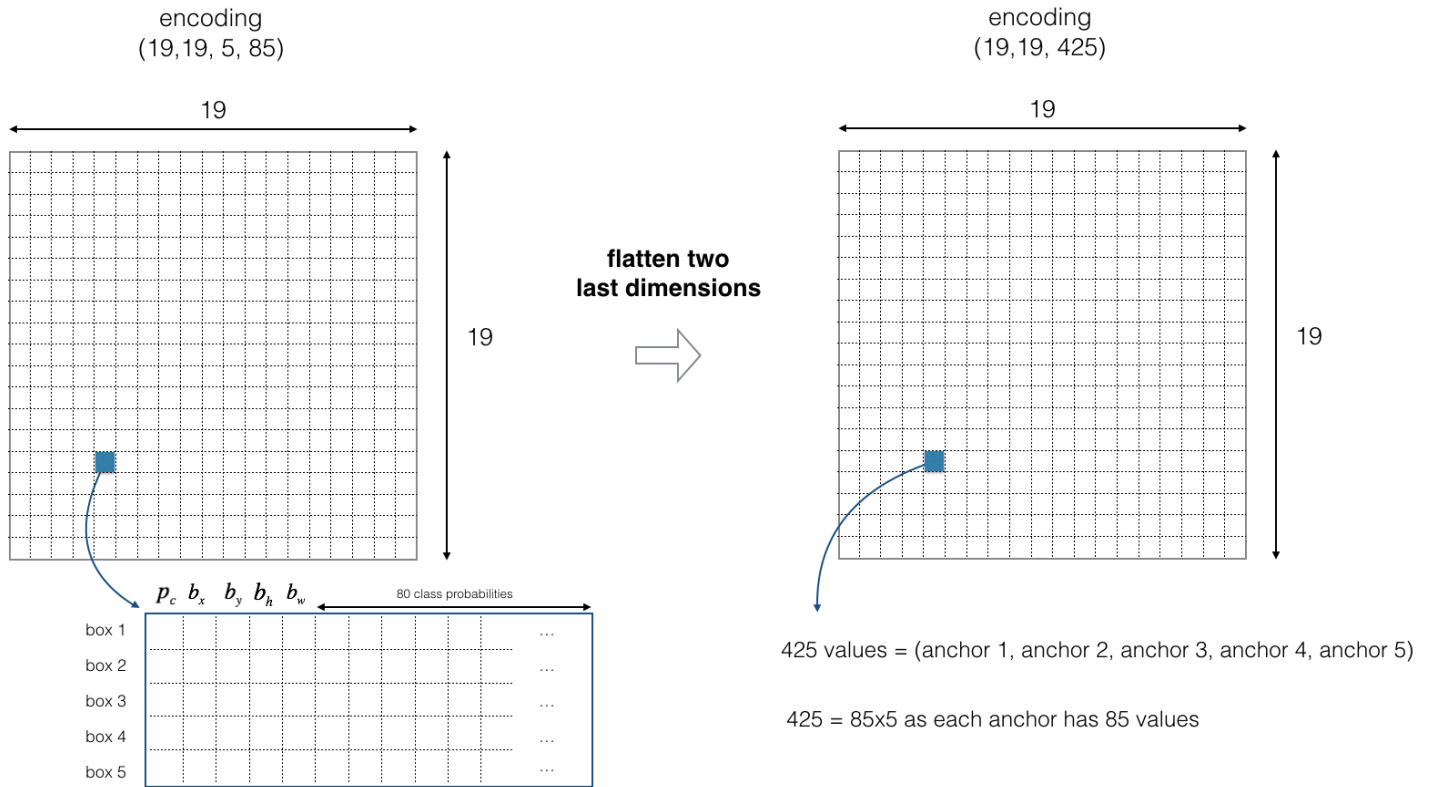
Let's look in greater detail at what this encoding represents.



If the center/midpoint of an object falls into a grid cell, that grid cell is responsible for detecting that object.

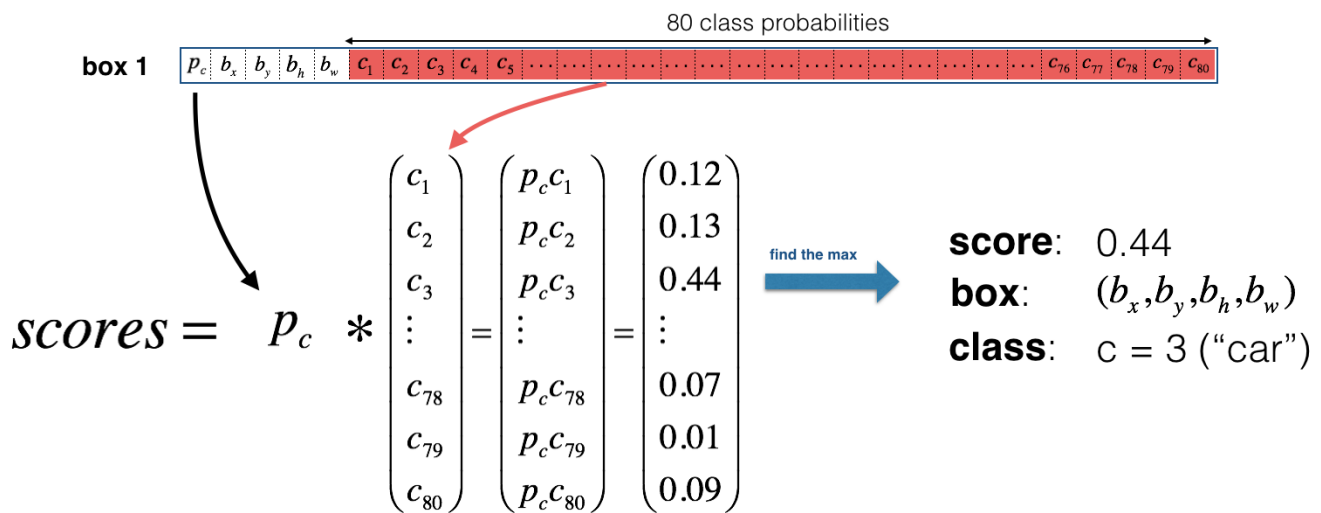
Since you're using 5 anchor boxes, each of the 19 x 19 cells thus encodes information about 5 boxes. Anchor boxes are defined only by their width and height.

For simplicity, you'll flatten the last two dimensions of the shape (19, 19, 5, 85) encoding, so the output of the Deep CNN is (19, 19, 425).



Class score

Now, for each box (of each cell) you'll compute the following element-wise product and extract a probability that the box contains a certain class. The class score is $\text{score}_{c,i} = p_c \times c_i$: the probability that there is an object p_c times the probability that the object is a certain class c_i .



the box (b_x, b_y, b_h, b_w) has detected $c = 3$ ("car") with probability score: 0.44

Example of figure:

- In the figure, let's say for box 1 (cell 1), the probability that an object exists is $p_1=0.60$. So, there's a 60% chance that an object exists in box 1 (cell 1).
- The probability that the object is the class "category 3 (a car)" is $c_3 = 0.73$.
- The score for box 1 and for category "3" is $\text{score}_{1,3}=0.60 \times 0.73=0.44$.

- Let's say you calculate the score for all 80 classes in box 1, and find that the score for the car class (class 3) is the maximum. So, you'll assign the score 0.44 and class "3" to this box "1".

Visualizing classes

Here's one way to visualize what YOLO is predicting on an image:

- For each of the 19x19 grid cells, find the maximum of the probability scores (taking a max across the 80 classes, one maximum for each of the 5 anchor boxes).
- Color that grid cell according to what object that grid cell considers the most likely.

Doing this results in this picture:



Note that this visualization isn't a core part of the YOLO algorithm itself for making predictions; it's just a nice way of visualizing an intermediate result of the algorithm.

Visualizing bounding boxes

Another way to visualize YOLO's output is to plot the bounding boxes that it outputs. Doing that results in a visualization like this:



Non-Max suppression

In the figure above, the only boxes plotted are ones for which the model had assigned a high probability, but this is still too many boxes. You'd like to reduce the algorithm's output to a much smaller number of detected objects.

To do so, you'll use **non-max suppression**. Specifically, you'll carry out these steps:

- Get rid of boxes with a low score. Meaning, the box is not very confident about detecting a class, either due to the low probability of any object, or low probability of this particular class.
- Select only one box when several boxes overlap with each other and detect the same object.

2.2 - Filtering with a Threshold on Class Scores

You're going to first apply a filter by thresholding, meaning you'll get rid of any box for which the class "score" is less than a chosen threshold.

The model gives you a total of 19x19x5x85 numbers, with each box described by 85 numbers. It's convenient to rearrange the (19,19,5,85) (or (19,19,425)) dimensional tensor into the following variables:

- `box_confidence` : tensor of shape (19, 19, 5, 1) containing p_c (confidence probability that there's some object) for each of the 5 boxes predicted in each of the 19x19 cells.
- `boxes` : tensor of shape (19, 19, 5, 4) containing the midpoint and dimensions (b_x, b_y, b_h, b_w) for each of the 5 boxes in each cell.
- `box_class_probs` : tensor of shape (19, 19, 5, 80) containing the "class probabilities" ($c_1, c_2, \dots, c_{80}$) for each of the 80 classes for each of the 5 boxes per cell.

Exercise 1 - yolo_filter_boxes

Implement `yolo_filter_boxes()` .

1. Compute box scores by doing the elementwise product as described in Figure 4 ($p \times c$).

The following code may help you choose the right operator:

```
a = np.random.randn(19, 19, 5, 1)
b = np.random.randn(19, 19, 5, 80)
c = a * b # shape of c will be (19, 19, 5, 80)
```

This is an example of **broadcasting** (multiplying vectors of different sizes).

2. For each box, find:

- the index of the class with the maximum box score
- the corresponding box score

Helpful Hints

- For the `axis` parameter of `argmax` and `reduce_max`, if you want to select the **last** axis, one way to do so is to set `axis=-1`. This is similar to Python array indexing, where you can select the last position of an array using `arrayname[-1]`.
- Applying `reduce_max` normally collapses the axis for which the maximum is applied. `keepdims=False` is the default option, and allows that dimension to be removed. You don't need to keep the last dimension after applying the maximum here.

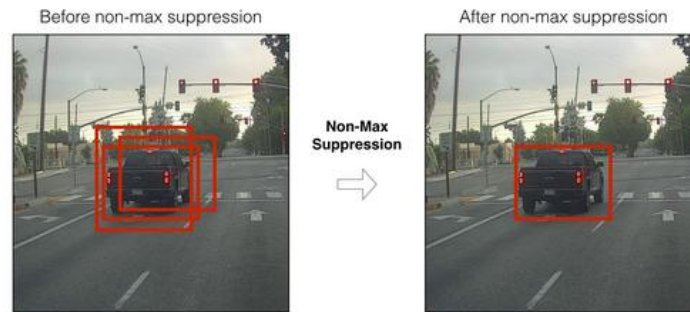
3. Create a mask by using a threshold. As a reminder: `([0.9, 0.3, 0.4, 0.5, 0.1] < 0.4)` returns: `[False, True, False, False, True]`. The mask should be `True` for the boxes you want to keep.
4. Use TensorFlow to apply the mask to `box_class_scores`, `boxes` and `box_classes` to filter out the boxes you don't want. You should be left with just the subset of boxes you want to keep.

Exercise 1 – TODO:

```
# See code from Canvas assignment
def yolo_filter_boxes(boxes, box_confidence, box_class_probs, threshold = .6):
    """Filters YOLO boxes by thresholding on object and class confidence.
```

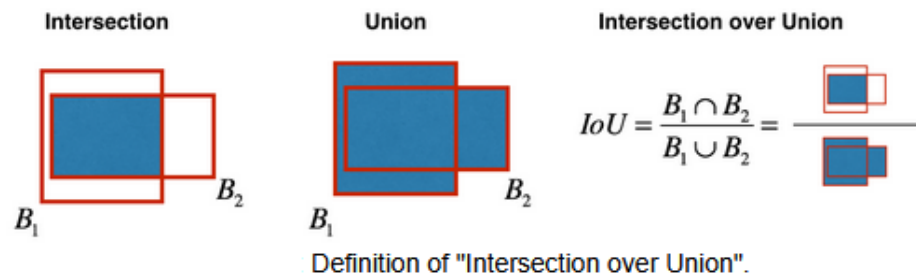
2.3 - Non-max Suppression

Even after filtering by thresholding over the class scores, you still end up with a lot of overlapping boxes. A second filter for selecting the right boxes is called non-maximum suppression (NMS).



In this example, the model has predicted 3 cars, but it's actually 3 predictions of the same car. Running non-max suppression (NMS) will select only the most accurate (highest probability) of the 3 boxes.

Non-max suppression uses the very important function called "**Intersection over Union**", or IoU.



Exercise 2 – TODO:

Implement `iou()`

Some hints:

- This code uses the convention that (0,0) is the top-left corner of an image, (1,0) is the upper-right corner, and (1,1) is the lower-right corner. In other words, the (0,0) origin starts at the top left corner of the image. As x increases, you move to the right. As y increases, you move down.
- For this exercise, a box is defined using its two corners: upper left (x_1, y_1) and lower right (x_2, y_2), instead of using the midpoint, height and width. This makes it a bit easier to calculate the intersection.
- To calculate the area of a rectangle, multiply its height ($y_2 - y_1$) by its width ($x_2 - x_1$). Since (x_1, y_1) is the top left and x_2, y_2 are the bottom right, these differences should be non-negative.
- To find the **intersection** of the two boxes (xi_1, yi_1, xi_2, yi_2):
 - Feel free to draw some examples on paper to clarify this conceptually.
 - The top left corner of the intersection (xi_1, yi_1) is found by comparing the top left corners (x_1, y_1) of the two boxes and finding a vertex that has an x-coordinate that is closer to the right, and y-coordinate that is closer to the bottom.
 - The bottom right corner of the intersection (xi_2, yi_2) is found by comparing the bottom right corners (x_2, y_2) of the two boxes and finding a vertex whose x-coordinate is closer to the left, and the y-coordinate that is closer to the top.
 - The two boxes **may have no intersection**. You can detect this if the intersection coordinates you calculate end up being the top right and/or bottom left corners of an intersection box. Another way to think of this is if you calculate the height ($y_2 - y_1$) or width ($x_2 - x_1$) and find that at least one of these lengths is negative, then there is no intersection (intersection area is zero).
 - The two boxes may intersect at the **edges or vertices**, in which case the intersection area is still zero. This happens when either the height or width (or both) of the calculated intersection is zero.

Additional Hints

- `xi1` = **maximum** of the `x1` coordinates of the two boxes
- `yi1` = **maximum** of the `y1` coordinates of the two boxes
- `xi2` = **minimum** of the `x2` coordinates of the two boxes
- `yi2` = **minimum** of the `y2` coordinates of the two boxes
- `inter_area` = You can use `max(height, 0)` and `max(width, 0)`

```
# See code from Canvas assignment
def iou(box1, box2):
    """Implement the intersection over union (IoU) between box1 and box2

    Arguments:
    box1 -- first box, list object with coordinates (box1_x1, box1_y1, box1_x2, box1_y2)
    box2 -- second box, list object with coordinates (box2_x1, box2_y1, box2_x2, box2_y2)
    """
```

2.4 - YOLO Non-max Suppression

You are now ready to implement non-max suppression. The key steps are as follows:

1. **Select the box with the highest score.**
2. **Compute the overlap** of this box with all other boxes of the same class. Remove any boxes that significantly overlap ($\text{IoU} \geq \text{iou_threshold}$).
3. **Repeat** the process: go back to step 1 and iterate until no remaining boxes have a score lower than the currently selected box.

This process effectively eliminates boxes that overlap significantly with the selected boxes, leaving only the "best" candidates.

For implementation, consider using a divide-and-conquer approach:

- **Divide** all the boxes by class.
- Apply non-max suppression for each class individually.
- Finally, **merge** the results from each class into a single set of boxes.

This approach is recommended for the upcoming exercise, as it simplifies the implementation while ensuring accuracy across multiple classes.

Exercise 3 - yolo_non_max_suppression

Implement `yolo_non_max_suppression()` using TensorFlow. TensorFlow has two built-in functions that are used to implement non-max suppression (so you don't actually need to use your `iou()` implementation):

Reference documentation:

- [tf.image.non_max_suppression\(\)](#)

```
tf.image.non_max_suppression(
    boxes, scores, max_output_size, iou_threshold=0.5,
    score_threshold=float('-inf'), name=None
)
```

Note: In the exercise below, for `tf.image.non_max_suppression()` you only need to set `boxes`, `scores`, `max_output_size` and `iou_threshold` parameters.

- [tf.gather\(\)](#)

```
tf.gather(
    params, indices, validate_indices=None, axis=None, batch_dims=0, name=None
)
```


Note: In the exercise below, for `tf.gather()` you only need to set `params` and `indices` parameters.

- [tf.boolean_mask\(\)](#)

```
tf.boolean_mask(  
    tensor, mask, axis=None, name='boolean_mask'  
)
```

Note: In the exercise below, for `tf.boolean_mask()` you only need to set `tensor` and `mask` parameters.

- [tf.concat\(\)](#)

```
tf.concat(  
    values, axis, name='concat'  
)
```

Exercise 3 – TODO:

```
# See code from Canvas assignment  
def yolo_non_max_suppression(scores, boxes, classes, max_boxes = 10, iou_threshold = 0.5):  
    """  
    Applies Non-max suppression (NMS) to set of boxes
```

2.5 - Wrapping Up the Filtering

It's time to implement a function taking the output of the deep CNN (the 19x19x5x85 dimensional encoding) and filtering through all the boxes using the functions you've just implemented.

Exercise 4 - yolo_eval

Implement `yolo_eval()` which takes the output of the YOLO encoding and filters the boxes using score threshold and NMS. There's just one last implementational detail you have to know. There're a few ways of representing boxes, such as via their corners or via their midpoint and height/width. YOLO converts between a few such formats at different times, using the following functions (which are provided):

```
boxes = yolo_boxes_to_corners(box_xy, box_wh)
```

which converts the yolo box coordinates (x,y,w,h) to box corners' coordinates (x1, y1, x2, y2) to fit the input of `yolo_filter_boxes`

```
boxes = scale_boxes(boxes, image_shape)
```

YOLO's network was trained to run on 608x608 images. If you are testing this data on a different size image -- for example, the car detection dataset had 720x1280 images -- this step rescales the boxes so that they can be plotted on top of the original 720x1280 image.

Don't worry about these two functions; you'll see where they need to be called below.

Exercise 4 – TODO:

```
# See code from Canvas assignment  
def yolo_eval(yolo_outputs, image_shape = (720, 1280), max_boxes=10, score_threshold=.6, iou_threshold=.5):
```

On Thursday, we'll test a pretrained YOLO model.